# Temporal Regularized Learning: Self-Supervised Learning Local in Space and Time

**Davide Wiest**
Department of Computer Science, TU Darmstadt
Darmstadt, Germany
November 2026

**Code:** https://github.com/Axym-Labs/TeReL **Paper:** https://arxiv.org/abs/2611.17019 **Correspondence:** dw@axym.org

**Abstract**

Layer-local self-supervised objectives remove the backward pass between layers, but almost all still compute their loss over a batch of samples and are therefore local in space but not in time. We introduce Temporal Regularized Learning (TEREL), a spatially and temporally local learning rule for self-supervised representation learning from sequential input streams. The key idea is to instantiate a VICReg-style objective with temporally adjacent positives and non-differentiated running population statistics, yielding a per-neuron online loss with bounded state. The objective is a soft form of Slow Feature Analysis, and its gradient is a Hebbian update of the feedforward weights and an anti-Hebbian update of one lateral matrix per layer. The aim of the experiments is to show that the rule works in a small network, not to compete with backpropagation. On a class-chunked MNIST stream, a two-layer fully connected network trained with batch size one outperforms random features (96.48% versus 95.09% linear-probe accuracy) and classical SFA, and a variant that relaxes both locality constraints is comparable to supervised backpropagation. Because training and inference use the same computation, the network can keep learning from unlabeled data during inference without loss of accuracy.

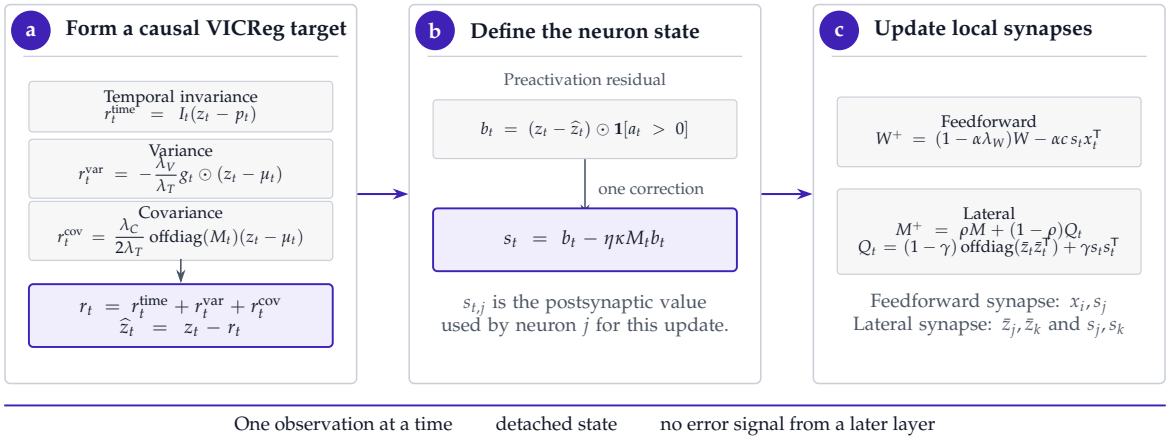



**Figure 1:** TEREL in one layer. **a**, The temporal, variance, and covariance terms form a regularized target. **b**, Its residual and one pass through the lateral matrix define the neuron state. **c**, The state drives a Hebbian feedforward and an anti-Hebbian lateral update.

## 1 Introduction

Learning from unlabeled sensory streams is a practical necessity for both biological modeling and energy-efficient machine learning on constrained hardware. Modern self-supervised methods suc-

ceed on unlabeled data by aligning representations of related samples (positives) while avoiding collapse using either negative samples or explicit regularizers. Contrastive methods rely on negatives and on large batch sizes to estimate cross-sample statistics [1, 2]. Regularized alternatives avoid explicit negatives and instead combine an invariance term with variance and covariance penalties [3, 4]. VICReg is a regularized method that enforces invariance without negatives and whose regularizers can be computed from local moments, at least in a batched implementation [3].

Standard implementations of these objectives use credit assignment across neurons and layers. That usually requires an extensive memory of the state for backpropagation [5], or iterative settling for predictive coding and equilibrium propagation [6, 7], which conflicts with temporal locality. For this reason, we optimize the objective at each neuron of the network. This paper shows how a VICReg-style objective can be instantiated as a per-neuron temporal loss by using adjacent observations as positives and treating running population statistics as non-differentiated state. The motivation is practical: locality reduces communication between units, and temporal locality permits streaming implementations with only a small short-term memory per neuron. Both properties are desirable for analog and neuromorphic hardware [8, 9].

By making the invariance temporal, TEREL is a type of Slow Feature Analysis (SFA) procedure: SFA aims to extract slowly changing features from a temporal sequence of faster-changing inputs [10, 11]. This can lead to representations that are invariant to augmentations or capture latent variables. More broadly, many sequential domains contain latent factors that evolve more slowly than their raw observations. In text, the sentiment of a review remains stable across its many words; in finance, stock prices fluctuate faster than the market factors that drive them; in physiology, the underlying state or health condition of a patient evolves slowly relative to heartbeat, respiratory cycles, and momentary noise in sensor traces.

The per-neuron loss gives the learning rule a familiar form. Its gradient at the preactivation of a neuron is a signed state; the feedforward update is the product of presynaptic activity and this state, and pairs of states drive an anti-Hebbian update of one same-layer matrix. Similarity-matching networks use the same form of Hebbian cross-layer and anti-Hebbian lateral learning for linear representations [12, 13]. The key difference is the more complex state used here: the residual of a nonlinear target that balances slowness, variance, and decorrelation.

**Contributions** This paper makes three contributions. First, it derives TEREL from VICReg through an explicit instantiation choice: population statistics are maintained as non-differentiated state, while adjacent observations supply the positive pairs, and these stop-gradients retain the learning signal of the current observation. Second, it expresses the resulting gradient as a neuron-local rule with Hebbian feedforward and anti-Hebbian lateral updates, and clarifies TEREL as a conceptual bridge between modern regularized self-supervised learning objectives and classical temporal representation learning ideas such as Slow Feature Analysis and trace learning [11, 14]. Third, it evaluates TEREL with batch size one under controlled temporal orderings against random, classical SFA, supervised, and relaxed-locality references, and reports ablations of each loss term.

The experiments test three hypotheses. (H1) Despite its locality constraints, TEREL trained with batch size one learns a useful deep representation. (H2) The temporal order and each of the three loss terms contribute to this representation. (H3) Continuing the same unlabeled updates along an evaluation stream preserves the representation, so that the rule can keep learning online.

## 2 Method

### 2.1 Locality

We call a learning rule *spatially local* if the update of a weight depends only on quantities available at its pre- and postsynaptic units (and same-layer auxiliary units) and does not require backpropagated error signals from other layers. We call it *temporally local* (online) if the gradient at time step $t$ depends only on the current observation, the current parameters, and a bounded amount of detached state carried forward in time. The size of this state may depend on the layer width but not on the length of the stream, and no gradient flows through the history that produced it.

### 2.2 Starting point: VICReg

Consider observation $t$ at a layer with input $x_t \in \mathbb{R}^d$ and width $D$. The layer computes

$$a_t = W_t x_t + q_t, \qquad z_t = \phi(a_t), \tag{1}$$

where $W_t \in \mathbb{R}^{D\times d}$ and $q_t \in \mathbb{R}^D$. The output $z_t$ is detached before it becomes the input to the next learned layer. Before this observation, each neuron stores its previous activation $p_t$, a running mean $\mu_t$, and a running variance $v_t$, and the layer stores a lateral matrix $M_t \in \mathbb{R}^{D\times D}$. Let $I_t \in \{0,1\}$ indicate whether $p_t$ belongs to the same sequence, and write $\bar{z}_t = z_t - \mu_t$ and $g_t = [\tau - v_t]_+$ for a variance target $\tau > 0$.

VICReg decomposes a representation-level objective into three terms (similarity/invariance, variance, covariance) [3]: $\mathcal{L}_{\text{VICReg}} = \lambda_I \mathcal{L}_I + \lambda_V \mathcal{L}_V + \lambda_C \mathcal{L}_C$. We keep these roles but replace the second view by the previous activation $p_t$, batch moments by the detached running statistics $\mu_t, v_t$, and the differentiated batch covariance by the lagged operator $A_t = \text{offdiag}(M_t)$:

$$\ell_T = \frac{I_t}{D}\|z_t - p_t\|_2^2, \qquad \ell_V = -\frac{1}{D} g_t^\top (\bar{z}_t \odot \bar{z}_t), \qquad \ell_C = \frac{1}{D} \bar{z}_t^\top A_t \,\text{sg}(\bar{z}_t). \tag{2}$$

The layer minimizes $\ell = \lambda_T \ell_T + \lambda_V \ell_V + \lambda_C \ell_C$ with $\lambda_T > 0$. The temporal coherence term makes consecutive activations similar; in discrete time it is the squared first derivative of the activations that SFA minimizes. The variance term increases the coordinates whose stored variance lies below $\tau$, and the covariance term applies a lagged same-layer decorrelation signal. TEREL is therefore a causal surrogate of the VICReg decomposition, not a re-expression of its batch objective: $\ell_V$ is a gated instantaneous expansion term, and $A_t$ is a lagged proxy for the covariance gradient.

**Proposition 1** (A stop-gradient VICReg instantiation is temporally local)**.** *For fixed $p_t, \mu_t, v_t, M_t$, and $I_t$, the derivative of $\ell$ is*

$$\nabla_{z_t} \ell = \frac{2\lambda_T}{D} r_t, \qquad r_t = I_t(z_t - p_t) - \frac{\lambda_V}{\lambda_T} g_t \odot \bar{z}_t + \frac{\lambda_C}{2\lambda_T} A_t \bar{z}_t, \tag{3}$$

*and the feedforward gradient is $2\lambda_T (J_{\phi,t}^\top r_t) x_t^\top / D$, where $J_{\phi,t}$ is the Jacobian of $\phi$ at $a_t$. The update is temporally local, and detachment preserves a learning signal from the current observation whenever $x_t \neq 0$ and $J_{\phi,t}^\top r_t \neq 0$.*

*Proof.* The stored quantities are constants for differentiation. The temporal, variance, and covariance terms contribute $2\lambda_T I_t(z_t - p_t)/D$, $-2\lambda_V g_t \odot \bar{z}_t / D$, and $\lambda_C A_t \bar{z}_t / D$, respectively, which sum to Equation (3). All of them are functions of $z_t$ and of quantities of size $O(D^2)$ computed before observation $t$, so no gradient reaches an earlier observation. The chain rule through Equation (1) gives the outer product $J_{\phi,t}^\top \nabla_{z_t} \ell \, x_t^\top$, which is nonzero when both of its factors are nonzero. □

### 2.3 Regularized target and neuron state

The residual $r_t$ induces an activation target, $\widehat{z}_t = \mathrm{sg}(z_t - r_t)$: updates move $z_t$ toward the previous activation, away from the running mean where the stored variance is below $\tau$, and against correlated activity. The local loss $\lambda_T \|z_t - \widehat{z}_t\|_2^2 / D$ has the same gradient with respect to $z_t$ as $\ell$. This target is formed within the layer; it is not an external target.

Expressed at the input side of the nonlinearity, the residual becomes $b_t = J_{\phi,t}^\top r_t$. For the ReLU encoders used in the experiments,

$$b_t = r_t \odot \mathbf{1}[a_t > 0], \tag{4}$$

which requires no automatic-differentiation graph. One pass through the lateral matrix gives the postsynaptic neuron state

$$s_t = b_t - \eta \kappa M_t b_t, \tag{5}$$

with correction step $\eta > 0$ and strength $\kappa \geq 0$. This is a single matrix–vector product, not an iterative settling procedure.

As in the general case, the weight update can be implemented in a Hebbian style, as the product of the input and an output-related term. With learning rate $\alpha$ and weight decay $\lambda_W$,

$$W_{t+1} = (1 - \alpha\lambda_W) W_t - \alpha \frac{2\lambda_T}{D} s_t x_t^\top, \tag{6}$$

which is the gradient step for the detached preactivation target $\widehat{a}_t = \mathrm{sg}(a_t - s_t)$. Each feedforward synapse therefore receives a presynaptic scalar $x_{t,i}$ and a postsynaptic scalar $s_{t,j}$, and under gradient descent the sign of $s_{t,j}$ determines potentiation or depression. The bias update uses $s_t$ alone. Without the lateral correction ($\kappa = 0$), Equation (6) is exactly the feedforward gradient of Proposition 1.

### 2.4 Lateral matrix and online memory requirements

The same matrix supplies the covariance signal in Equation (3) and the state correction in Equation (5). After the synaptic update, it follows

$$M_{t+1} = \rho M_t + (1 - \rho)\left[(1 - \omega)\,\mathrm{offdiag}(\bar{z}_t \bar{z}_t^\top) + \omega s_t s_t^\top\right], \tag{7}$$

with $\omega \in [0, 1]$ and $\rho \in [0, 1)$. With Hebbian-style updates, mean-centered products between pairs of neurons recover covariance over time. If the inhibitory lateral weights are written as $L_t = -M_t$, the state component contributes $-(1 - \rho)\omega s_t s_t^\top$ to $L_t$: same-signed neuron states make their mutual connection more inhibitory, which is the anti-Hebbian sign.

The TEREL loss requires three per-neuron state variables: the previous activation, a running mean, and a running variance,

$$p_{t+1} = z_t, \qquad \mu_{t+1} = \beta\mu_t + (1 - \beta) z_t, \qquad v_{t+1} = \beta v_t + (1 - \beta)(z_t - \mu_t)^2. \tag{8}$$

This state can be stored at the neuron level. The running statistics, the lateral matrix Equation (7), and the feedforward weights Equation (6) are all leaky integrators (exponential moving averages); an RC circuit could, in principle, implement such leaky integrators. At a sequence boundary only the validity of $p_t$ is cleared. A layer of width $D$ thus stores $3D$ dynamic scalars and $D^2$ auxiliary lateral parameters besides its feedforward weights; plain SGD adds no optimizer state. For the 784–512–256 MNIST encoder these are 2,304 dynamic scalars, 327,680 lateral parameters, and 533,248 feedforward parameters.

TEREL does not attempt to approximate global backpropagation across long paths in the network. Because the learning objective is local in space and time, the method also does not accumulate approximation error with depth in the way that methods approximating global gradients do. Each layer completes its forward pass, target, state correction, and updates for observation $t$ in one feedforward schedule.

### 2.5 TEREL-Offline

TEREL-Offline relaxes the temporal and spatial locality constraints within a subsequence of the data stream. A subsequence enters a ReLU multilayer perceptron, the same three-term objective is evaluated at its final layer, and backpropagation differentiates it through all layers and observations of the subsequence. It is optimized with plain SGD and serves as a performance reference for the objective, not as an alternative definition of TEREL.

## 3 Related work

TEREL occupies a point in design space at the intersection of self-supervised and local learning procedures. We summarize the most direct comparisons as follows:

- **Contrastive SSL:** Compared to methods that rely on negative samples, TEREL requires only temporally adjacent positives and regularizers to avoid collapse [1, 2].
- **VICReg and Barlow Twins:** TEREL moves the variance and covariance regularization to the neuron level and substitutes temporally adjacent positives for augmented pairs [3, 4].
- **Layerwise SSL:** Layer-local and blockwise objectives remove the backward path between network modules [15–18], and VICReg losses have been applied at successive layers [19]. TEREL differs by additionally targeting temporal locality and online implementability: it supplies the three signals causally along time and factors them into neuron-local updates.
- **Similarity matching:** Hebbian/anti-Hebbian networks derived from similarity-matching objectives have the same synaptic form [12, 13]; TEREL replaces their linear postsynaptic factor by the residual of a nonlinear regularized target.
- **Forward-Forward:** TEREL and Forward-Forward both use local, layerwise objectives to avoid end-to-end backpropagation through deep networks. TEREL uses an invariance objective rather than a goodness signal and does not require constructed positive and negative passes [20].
- **Equilibrium propagation and predictive coding:** These rely on iterative relaxation, and equilibrium propagation on a free and a nudged phase, whereas TEREL uses a single-pass, per-neuron objective with bounded local state [6, 7, 21].

Vectors and matrices that are computed from parameters or activations but held constant with respect to the parameters being optimized are common in training objectives, such as weight-averaged teacher parameters [22], stop-gradient targets [23, 24], predictions of an earlier model [25], and importance matrices in continual learning [26, 27]. TEREL relies on this design pattern through its running variance and covariance estimates.

**Relation to slow feature analysis and trace learning** TEREL can be seen as a regularized, modernized trace-learning/SFA-style procedure in which temporal adjacency provides the positive pairs. The MSE between two contiguous samples is exactly the first derivative of activations in discrete time that is squared and minimized in SFA, and the whitening constraint of SFA maps to the variance and covariance regularizers [10]. In the classification experiments, the faster-changing inputs are the individual samples, while the slower feature is the class membership of all samples within a chunk. Incremental SFA adapts the linear construction to streams but retains a whitening stage and linear feature extraction [28]. TEREL is also closely related to Bio-SFA, a biologically plausible SFA algorithm. Bio-SFA works with a hard whitening constraint and uses iterative settling, whereas TEREL penalizes covariance softly and its training is a series of discrete-time passes [29].

Trace learning is a simple, biologically plausible procedure that modifies Hebbian updates by introducing a temporal aspect: instead of a neuron's activation, it uses an activation trace [11, 30]. Trace learning and TEREL share the operational goal of promoting temporal invariance but do so in different ways. Ignoring shared factors, the temporal term of TEREL induces $\Delta w_{kj} \propto x_k(z_{t-1,j} - z_{t,j})$,

while trace learning uses $\Delta w_{kj} \propto x_k(\beta \bar{z}_{t-1,j} + (1-\beta) z_{t,j})$ with $\beta \in (0,1)$. No choice of $\beta$ makes the two updates identical, because matching them would require both $\beta = 1$ and $1 - \beta = -1$. The trace rule furthermore requires a weight-dependent decay term to prevent runaway effects [31].

More broadly, TEREL opens a bridge between modern self-supervised learning and classical local learning rules by showing how regularized objectives can be expressed in strictly local, online-compatible form. Our ablations show that the variance and decorrelation regularizers are both necessary to avoid collapse or divergence. This suggests that trace-learning and SFA-style rules may likewise benefit from explicit variance control and decorrelation, even if their core temporal objective remains unchanged.

## 4 Experimental protocol

We evaluate TEREL on MNIST [32] with synthetic sequential coherence. TEREL's objective is strictly representational and uses no supervision. However, the order of the data stream can induce relatively arbitrary pairs of positives. For example, placing data of the same category next to each other mirrors a supervised contrastive setup. We use this order: the stream consists of randomly permuted class-homogeneous chunks of 16 samples, separated by sequence boundaries. The labels determine adjacency but do not enter the target or either update. As noted by Hinton for the supervised Forward-Forward setup, the network should then preferentially learn features that support discrimination between classes [20].

The encoder is a ReLU multilayer perceptron with hidden widths 512 and 256. TEREL is trained with batch size one and plain SGD for two passes over the encoder-training samples. The downstream evaluation trains a linear classifier on the standardized concatenated activations of both layers of the encoder-training samples and evaluates it on held-out data. We reserve a stratified subset of the official training data (10,000 images) for all model selection; the official test split is used only for the final comparison. Reported values are means and sample standard deviations over five initializations, and paired comparisons use 95% Student-$t$ intervals. All trained models, including the linear classifier and the references, use plain SGD; batch size is part of each method. Hyperparameters and raw values appear in Appendix A.

The references expose distinct relaxations. TEREL-Offline uses the same architecture and objective with subsequences of 128 and end-to-end backpropagation. A layer-local network trained with a supervised contrastive objective [33] (Layer-local SupCon) uses labels, minibatches, and batch normalization at every layer. A supervised offline network is trained end to end with labels and backpropagation. A random ReLU encoder of the same shape, a linear probe on the input, and batch and incremental SFA [10, 28] complete the comparison. The methods differ in their intrinsic data and optimization requirements, so the comparison reports representation quality rather than equal compute.

Four ablations examine TEREL itself: shuffling the stream, and setting the temporal, variance, or covariance coefficient to zero. For the online learning demonstration, the trained encoder continues its unlabeled updates along the evaluation stream, using the same fitted probe as the fixed encoder. Each representation is read before the current observation updates the encoder. The learning rate of these continued updates is a fixed fraction of the training learning rate, chosen on the validation stream.

## 5 Results

The aim of the experiments is to show that TEREL works in relatively small networks under its locality constraints, not to compete with architectures that rely on global credit assignment. As in the

Forward-Forward experiments [20], we therefore use networks without weight sharing. A shared weight would receive the sum of updates from many neurons, so its update would no longer be local to one synapse.

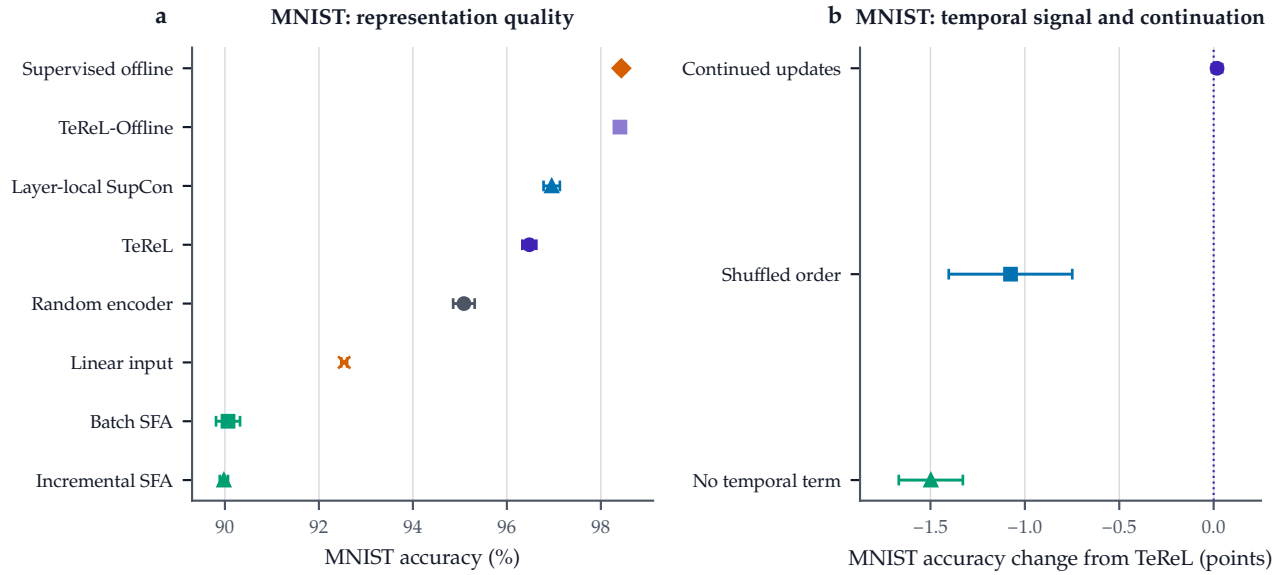



**Figure 2: Representation quality and paired temporal effects on MNIST. a**, Held-out linear-probe accuracy (mean and sample standard deviation). **b**, Paired accuracy changes relative to TEREL with a fixed encoder, with 95% Student-$t$ intervals.

### 5.1 Representation quality (H1)

TEREL reaches $96.48 \pm 0.15\%$ held-out accuracy on MNIST (Figure 2a). The random encoder reaches $95.09 \pm 0.23\%$, a paired gain of 1.39 percentage points $[1.04, 1.74]$, and the gain over a linear probe on the input is 3.94 points $[3.76, 4.12]$. Batch and incremental SFA reach $90.07 \pm 0.26\%$ and $89.98 \pm 0.09\%$. Layer-local SupCon reaches $96.95 \pm 0.17\%$, 0.48 points $[0.10, 0.85]$ above TEREL, with labels and mini-batches at every layer. TEREL-Offline reaches $98.41 \pm 0.04\%$, on par with the fully supervised offline network at $98.44 \pm 0.09\%$ (paired difference $-0.03$ points $[-0.19, 0.13]$). The objective itself therefore supports supervised-level accuracy when cross-layer and cross-time optimization are available; the gap to TEREL is the cost of the locality constraints.

### 5.2 Temporal order and loss terms (H2)

Removing the temporal order or any single loss term degrades the representation (Figure 2b). Shuffling the stream lowers accuracy by 1.08 paired points $[0.75, 1.40]$, and removing the temporal term lowers it by 1.50 points $[1.33, 1.67]$. Without the variance term, the representation collapses: the median feature variance falls to about $1.4 \times 10^{-5}$ and accuracy to $15.36 \pm 0.27\%$. Without the covariance term, training diverges for all five initializations before a representation can be measured. As in VICReg [3], the invariance term alone admits trivial solutions; the variance term prevents a constant representation, and the covariance term keeps the multidimensional code stable.

### 5.3 Online learning demonstration (H3)

Because training and inference use the same per-sample computation, the encoder can keep learning along the evaluation stream. With the training step size, these continued updates are too aggressive for an already trained representation: on the validation stream, the loss grows across the stream and the effective rank drops sharply. Scaling the step by $10^{-3}$ preserves validation accuracy (96.51% versus 96.48% for the fixed encoder) and effective rank. With this scale fixed, continued updates on the held-out stream reach $96.50 \pm 0.15\%$, compared with $96.48 \pm 0.15\%$ for the fixed encoder, a paired change of $+0.02$ points $[-0.01, 0.04]$, while the feedforward weights keep changing ($\|\Delta W\|_2 = 0.047$).

### 5.4 Representations

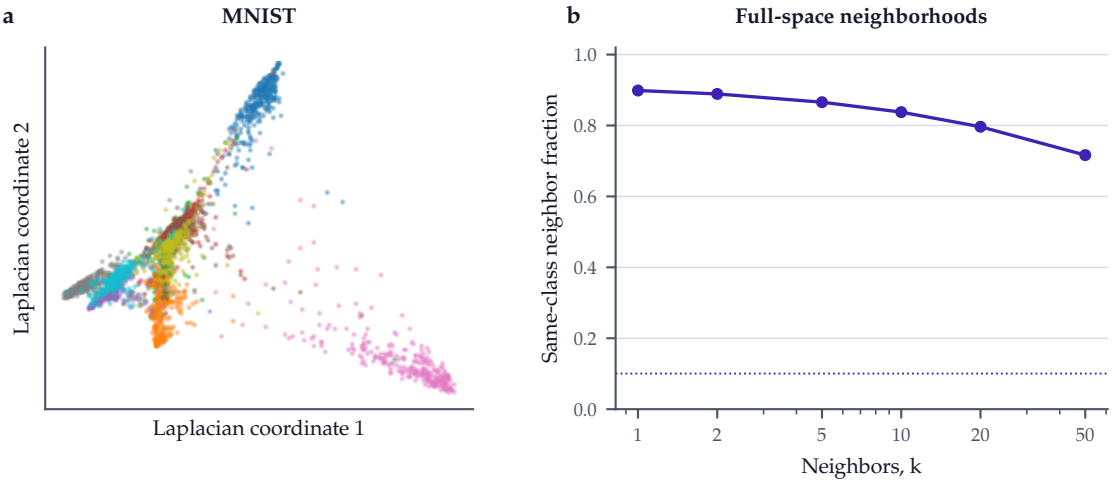



**Figure 3: Class structure in the MNIST TEREL representation. a**, Laplacian embedding of validation features of both layers; colors denote digits and are used only for visualization. **b**, Fraction of same-class neighbors in the complete 768-dimensional representation. The dotted line is the chance level.

We probe whether TEREL yields class-structured latent codes suitable for a simple linear readout. The Laplacian embedding in Figure 3a shows class-consistent branches rather than a single collapsed mass. Because a two-dimensional projection can distort neighborhoods, Figure 3b measures the same relation in the complete representation: the nearest neighbor shares the digit for 89.9% of observations, and the fraction remains 79.7% across 20 neighbors and 71.7% across 50, compared with a 10.0% chance level.

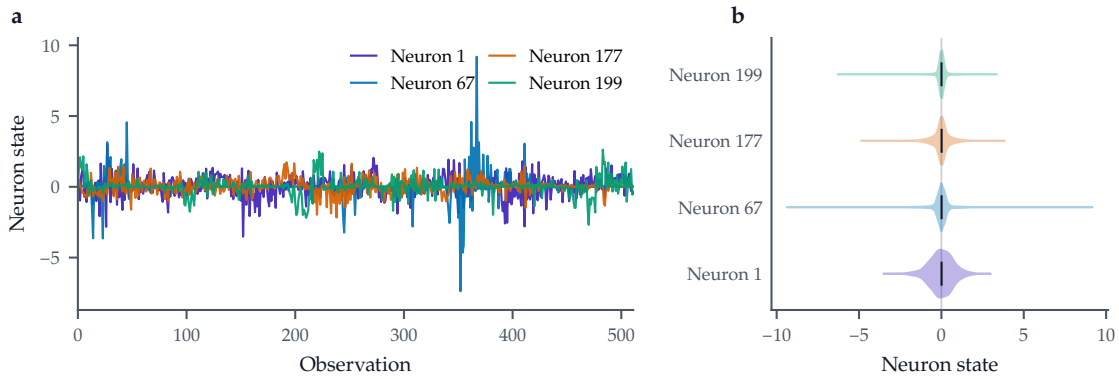



**Figure 4: Signed postsynaptic state. a**, State trajectories of four randomly selected second-layer neurons over the same interval of the validation stream. **b**, Their state distributions over the complete validation stream; black bars mark medians.

Figure 4 shows TEREL's neuron state $s_t$, the quantity that multiplies presynaptic activity in Equation (6). The selected neurons alternate between positive and negative values, so the state drives both potentiation and depression. Their scales differ substantially: some neurons respond through sparse excursions, whereas another carries a broader signed signal.

## 6 Limitations and future work

The experiments use a constructed, label-ordered stream on MNIST. Natural streams contain several time scales and may require a different balance among the temporal, variance, and covariance terms; establishing TEREL on video or sensor streams is the most important empirical extension. Continued online updates require a smaller step than initial training, and in the present setting they preserve rather than improve the representation; local rules that adapt this step would let training and long-term inference coincide more fully. Finally, a layer of width $D$ uses $D^2$ lateral parameters, which may dominate very wide layers. Sparse, low-rank, or structured lateral matrices, as well as convolutional and recurrent architectures, are natural next steps for constrained hardware.

## 7 Conclusion and outlook

We presented Temporal Regularized Learning, an approach that adapts the invariance, variance, and covariance regularizers of VICReg to neuron-level temporal losses. Detached running statistics and a lagged lateral matrix make the rule local in space and time, and its gradient takes the form of Hebbian feedforward and anti-Hebbian lateral updates driven by a signed neuron state. Trained with batch size one, TEREL learns representations that improve on random features and classical SFA, and the same rule keeps learning online after training. TEREL emphasizes locality and streaming compatibility and therefore maps naturally to constrained hardware, while connecting regularized self-supervised learning to classical temporal ideas such as Slow Feature Analysis and trace learning.

## AI Use Statement

An earlier version of TEREL and of this manuscript predates the generative-AI assistance. Generative AI tools were used primarily to update the paper with new experiments, to find bugs, and for script-

ing. They also assisted with analysis, figure creation, and editing. The author reviewed AI-assisted code, experimental outputs, numerical claims, citations, figures, and text against saved artifacts and executable code, and takes responsibility for the final content.

## References


[1] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A Simple Framework for Contrastive Learning of Visual Representations. *arXiv preprint arXiv:2002.05709*, 2020.

[2] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. Representation Learning with Contrastive Predictive Coding. *arXiv preprint arXiv:1807.03748*, 2018.

[3] Adrien Bardes, Jean Ponce, and Yann LeCun. VICReg: Variance-Invariance-Covariance Regularization for Self-Supervised Learning. In *International Conference on Learning Representations*, 2022. URL https://openreview.net/forum?id=xm6YD62D1Ub.

[4] Jure Zbontar, Li Jing, Ishan Misra, Yann LeCun, and Stéphane Deny. Barlow Twins: Self-Supervised Learning via Redundancy Reduction. *arXiv preprint arXiv:2103.03230*, 2021.

[5] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. Learning representations by back-propagating errors. *nature*, 323(6088):533–536, 1986.

[6] Rajesh P. N. Rao and Dana H. Ballard. Predictive Coding in the Visual Cortex: A Functional Interpretation of Some Extra-Classical Receptive-Field Effects. *Nature Neuroscience*, 2(1):79–87, 1999. doi: 10.1038/4580.

[7] Benjamin Scellier and Yoshua Bengio. Equilibrium propagation: Bridging the gap between energy-based models and backpropagation. *Frontiers in computational neuroscience*, 11:24, 2017.

[8] Giacomo Indiveri and Shih-Chii Liu. Memory and information processing in neuromorphic systems. *Proceedings of the IEEE*, 103(8):1379–1397, 2015.

[9] Mike Davies, Narayan Srinivasa, Tsung-Han Lin, Gautham Chinya, Yongqiang Cao, Sri Harsha Choday, Georgios Dimou, Prasad Joshi, Nabil Maghazeh, Shirley Miao, and other. Loihi: A neuromorphic manycore processor with on-chip learning. *IEEE Micro*, 38(1):82–99, 2018.

[10] Laurenz Wiskott and Terrence J Sejnowski. Slow feature analysis: Unsupervised learning of invariances. *Neural computation*, 14(4):715–770, 2002.

[11] P Földiák. Learning invariance from transformation sequences. *Neural computation*, 3(2):194–200, 1991.

[12] Cengiz Pehlevan, Tao Hu, and Dmitri B. Chklovskii. A Hebbian/Anti-Hebbian Neural Network for Linear Subspace Learning: A Derivation from Multidimensional Scaling of Streaming Data. *Neural Computation*, 27(7):1461–1495, 2015. doi: 10.1162/NECO_a_00745.

[13] Cengiz Pehlevan, Anirvan M. Sengupta, and Dmitri B. Chklovskii. Why Do Similarity Matching Objectives Lead to Hebbian/Anti-Hebbian Networks? *Neural Computation*, 30(1):84–124, 2018. doi: 10.1162/neco_a_01018.

[14] Erkki Oja. A simplified neuron model as a principal component analyzer. *Journal of mathematical biology*, 15(3):267–273, 1982.

[15] Sindy Löwe, Peter O'Connor, and Bastiaan S. Veeling. Putting An End to End-to-End: Gradient-Isolated Learning of Representations. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[16] Yuwen Xiong, Mengye Ren, and Raquel Urtasun. LoCo: Local Contrastive Representation Learning. In *Advances in Neural Information Processing Systems*, volume 33, 2020.

[17] Bernd Illing, Jean Ventura, Guillaume Bellec, and Wulfram Gerstner. Local plasticity rules can learn deep representations using self-supervised contrastive predictions. In *Advances in Neural Information Processing Systems*, volume 34, 2021.

[18] Shoaib Ahmed Siddiqui, David Krueger, Yann LeCun, and Stéphane Deny. Blockwise Self-Supervised Learning at Scale. *Transactions on Machine Learning Research*, 2024.

[19] Joy Datta, Rawhatur Rabbi, Puja Saha, Aniqua Nusrat Zereen, M. Abdullah Al-Wadud, and Jia Uddin. Deep Representation Learning Using Layer-Wise VICReg Losses. *Scientific Reports*, 15: 27049, 2025. doi: 10.1038/s41598-025-08504-2.

[20] Geoffrey E Hinton. The Forward-Forward Algorithm: Some Preliminary Investigations. *arXiv preprint arXiv:2212.13345*, 2022.

[21] Yann LeCun, Sumit Chopra, Raia Hadsell, Marc'Aurelio Ranzato, and Fu Jie Huang. A tutorial on energy-based learning. *Predicting structured data*, 1(0), 2006.

[22] Antti Tarvainen and Harri Valpola. Mean Teachers Are Better Role Models: Weight-Averaged Consistency Targets Improve Semi-Supervised Deep Learning Results. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[23] Jean-Bastien Grill, Florian Strub, Florent Altché, Corentin Tallec, Pierre H. Richemond, Elena Buchatskaya, Carl Doersch, Bernardo Avila Pires, Zhaohan Daniel Guo, Mohammad Gheshlaghi Azar, Bilal Piot, koray kavukcuoglu, Remi Munos, and Michal Valko. Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning. *arXiv preprint arXiv:2006.07733*, 2020.

[24] Xinlei Chen and Kaiming He. Exploring Simple Siamese Representation Learning. *arXiv preprint arXiv:2011.10566*, 2021.

[25] Zhizhong Li and Derek Hoiem. Learning without Forgetting. In *European Conference on Computer Vision*, 2016.

[26] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming Catastrophic Forgetting in Neural Networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.

[27] Davide Wiest. Improved Elastic Weight Consolidation as an Optimization Constraint for Continual Learning. *arXiv preprint arXiv:2611.16039*, 2026.

[28] Varun Raj Kompella, Matthew Luciw, and Jürgen Schmidhuber. Incremental Slow Feature Analysis: Adaptive and Episodic Learning from High-Dimensional Input Streams. *Neural Computation*, 24(11):2994–3024, 2012. doi: 10.1162/NECO_a_00344.

[29] David Lipshutz, Charles Windolf, Siavash Golkar, and Dmitri Chklovskii. A Biologically Plausible Neural Network for Slow Feature Analysis. In *Advances in Neural Information Processing Systems*, volume 33, pages 14986–14996, 2020.

[30] Simon M Stringer, G Perry, Edmund T Rolls, and JH Proske. Invariant object recognition in the visual system with novel views of 3D objects. *Neural Networks*, 15(10):1235–1246, 2002.

[31] Peter Földiák. Forming sparse representations by local anti-Hebbian learning. *Biological cybernetics*, 64(2):165–170, 1990.

[32] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[33] Prannay Khosla, Piotr Teterwak, Chen Wang, Aaron Sarna, Yonglong Tian, Phillip Isola, Aaron Maschinot, Ce Liu, and Dilip Krishnan. Supervised Contrastive Learning. In *Advances in Neural Information Processing Systems*, volume 33, pages 18661–18673, 2020.

# A Evaluation protocol and raw values

Table 1 lists the reported configurations. All hyperparameters are selected on the reserved validation split; the test split is used only for the final comparison.

**Table 1:** Encoder configurations used in the reported comparison. A sample is one observation presented to the encoder.

| Method | Samples | Batch or subsequence | Optimizer | Normalization |
|---|---|---|---|---|
| TEREL | 100,000 | 1 | SGD | none |
| TEREL-Offline | 1,000,000 | 128 | SGD | none |
| Layer-local SupCon | 3,000,000 | 256 | SGD | BatchNorm |
| Supervised offline | 500,000 | 64 | SGD | none |
| Batch SFA | 50,000 | complete fit | eigensolver | whitening |
| Incremental SFA | 50,000 | 1 | incremental | whitening |

For TEREL, the selected coefficients are $\lambda_T = 1$, $\lambda_V = 3.001981$, $\lambda_C = 0.606133$, $\tau = 1$, $\beta = 0.99$, $\rho = 0.995$, $\kappa = 0.574950$, $\eta = 0.1$, and $\omega = 0.591110$. The learning rate is 0.0138703 with weight decay $10^{-6}$. TEREL-Offline uses 20 epochs, learning rate 0.4, weight decay $10^{-6}$, variance coefficient 2.529924, covariance coefficient 0.0218471, and variance target 0.862789. The supervised offline network uses learning rate 0.3 and Layer-local SupCon learning rate 0.3. The linear probe is trained for 60 epochs with batch size 1024 on features standardized with training statistics; its learning rate is selected per method on the validation split from $\{0.01, 0.03, 0.1, 0.3, 1\}$ and is 0.3 for TEREL and its ablations.

# B Locality and operation accounting

For a layer of width $D$ with input width $d$, the feedforward multiplication costs $Dd$ multiply–accumulates. Forming the target and running statistics is linear in $D$. The lateral target and state correction each use a $D \times D$ matrix–vector multiplication, and updating the lateral moment uses a rank-one outer product. The persistent quantities are summarized in Table 3; temporary activations are not persistent state.

**Table 2:** Held-out MNIST accuracy by seed. Values are fractions; a dash denotes diverged training.

| Method / seed | 1101 | 1202 | 1303 | 1404 | 1505 |
|---|---|---|---|---|---|
| TEREL | .9637 | .9647 | .9671 | .9633 | .9651 |
| TEREL-Offline | .9847 | .9837 | .9841 | .9836 | .9842 |
| Random encoder | .9475 | .9529 | .9495 | .9520 | .9524 |
| Input linear probe | .9254 | .9262 | .9257 | .9250 | .9246 |
| Supervised offline | .9836 | .9858 | .9840 | .9846 | .9838 |
| Layer-local SupCon | .9698 | .9713 | .9677 | .9711 | .9678 |
| Batch SFA | .9039 | .9010 | .8981 | .8981 | .9022 |
| Incremental SFA | .8984 | .8998 | .9001 | .9008 | .8998 |
| No temporal term | .9506 | .9503 | .9503 | .9478 | .9500 |
| No variance term | .1530 | .1537 | .1582 | .1515 | .1518 |
| No covariance term | – | – | – | – | – |
| Shuffled order | .9559 | .9512 | .9535 | .9539 | .9556 |

**Table 3:** Persistent quantities for one TEREL layer of width $D$ and input width $d$.

| Category | Quantity | Size |
|---|---|---|
| Dynamic neuron state | $p, \mu, v$ | $3D$ |
| Auxiliary lateral parameters | $M$ | $D^2$ |
| Feedforward parameters | $W, q$ | $Dd + D$ |
| Optimizer state | none | 0 |